

Aplicações e Serviços Web 2025/2026

TP7: Autenticação e Segurança de APIs

1. Instalação e Configuração

Para começar, precisamos de instalar as bibliotecas necessárias

Nota: Para este guião irá necessitar de ter todas as ferramentas dos guiões anteriores já instaladas. Pode ser útil copiar os ficheiros package.json e package-lock.json e de seguida correr o npm install caso esteja a usar uma nova pasta para este guião, desta forma todos os package irão ser instalados.

1. De seguida deve executar os seguintes comandos no terminal para instalar as ferramentas:

```
npm install helmet cors express-rate-limit jsonwebtoken bcryptjs  
npm install -D @types/jsonwebtoken @types/bcryptjs
```

2. Segurança Global da API

Antes de protegermos as rotas individualmente, devemos proteger o servidor contra ataques automatizados simples. Para isso vamos utilizar o **helmet** para configurar cabeçalhos HTTP, o **cors** para definir quem pode aceder à API num browser, e o **express-rate-limit** para prevenir ataques de *distributed denial of service* (DDoS).

1. No seu ficheiro **src/server.ts**, adicione as seguintes configurações:

```
import helmet from 'helmet';  
import cors from 'cors';  
import rateLimit from 'express-rate-limit';  
  
// 1. Ocultar metadados do servidor e mitigar XSS  
app.use(helmet());  
  
// 2. Permitir apenas pedidos de origens confiáveis (ex: o frontend)  
app.use(cors({  
  origin: ['http://localhost:5173'], // corresponde ao endereço de origem do pedido  
  methods: ['GET', 'POST', 'PUT', 'DELETE']  
}));  
  
// 3. Limitar o número de pedidos para prevenir Força Bruta e DDoS  
const limiter = rateLimit({  
  windowMs: 15 * 60 * 1000, // Janela de 15 minutos
```

```

max: 100, // Limite de 100 pedidos por IP
message: { error: "Demasiados pedidos a partir deste IP, tente novamente mais
tarde." }
});
app.use(limiter);

```

2. No terminal, execute o script para ligar o servidor:

npm run dev

3. Realize vários pedidos de forma a verificar se os mecanismos de segurança estão ativos, deverá utilizar o curl ou Thunder Client (vscode).

- a. Helmet: Realize um pedido GET e inspecione os Headers da resposta. Verifique a ausência do header X-Powered-By: Express e a presença de Content-Security-Policy

- b. CORS: Crie um ficheiro **test-cors.html** no seu ambiente de trabalho (fora do projeto) com o seguinte código:

```

<!DOCTYPE html>
<html>
<body>
  <button onclick="fetch('http://localhost:3000/api/users').then(res =>
console.log('Sucesso')).catch(err => console.error('Erro de CORS:', err))">
    Forçar Chamada à API
  </button>
</body>
</html>

```

- i. Abra este ficheiro no seu browser com o Live Server (vscode).
- ii. Abra as Ferramentas de Programador (pressionando F12) e navegue para o separador Consola (Console).
- iii. Clique no botão. Deverá observar um erro a vermelho (CORS Policy), demonstrando que o browser abortou a comunicação porque a origem do ficheiro não está na lista de permissões (cors) configurada no seu server.ts.
- iv. Volte ao ficheiro src/server.ts na sua API para adicionar o servidor que está a correr a página html (Live Server):

```

app.use(cors({
  origin: ['http://localhost:5173', 'http://127.0.0.1:5500'], // Altere para a
  porta do seu Live Server
  methods: ['GET', 'POST', 'PUT', 'DELETE']
}));

```
- v. Volte a abrir o ficheiro seu browser com o Live Server (vscode)
- vi. Agora mensagem Sucesso! aparecerá na consola, provando que o browser reconheceu a origem como legítima.

- c. Rate Limit: Reduza o limite de pedidos para 2 (apenas para efeitos de teste), e de seguida realize pedidos sucessivos no browser ou Swagger. Após o 3º pedido, deverá receber o status 429 Too Many Requests

3. Autenticação com JWT e password hashing com Bcrypt

Para suportar o login, o nosso modelo de utilizador necessita de armazenar uma palavra-passe. **Atenção:** As senhas nunca devem ser guardadas em claro.

1. Comece por atualizar o seu modelo utilizador no schema.prisma:

```
model User {  
  id      Int      @id @default(autoincrement())  
  fullName String  
  email   String   @unique  
  password String? //o ? indica que o campo password é opcional  
  createdAt DateTime @default(now())  
}
```

2. Execute a migração para a base de dados:

```
npx prisma migrate dev --name add_password  
npx prisma generate
```

3. No ficheiro **src/services/userService.ts**, atualize a função de criação para aplicar o algoritmo de Hash antes de guardar na base de dados:

```
import bcrypt from 'bcryptjs';  
  
export const createUser = async (fullName: string, email: string, password: string) => {  
  const hashedPassword = await bcrypt.hash(password, 10);  
  
  return await prisma.user.create({  
    data: { fullName, email, password: hashedPassword }  
  });  
};
```

4. Deverá atualizar também o seu **userSchema.ts** (Zod) e o **userController.ts** para exigirem o envio do campo password no req.body:

```
password: z  
  .string()  
  .min(8, "A password deve ter pelo menos 8 caracteres")  
  .regex(/[A-Z]/, "Deve conter pelo menos uma letra maiúscula")  
  .regex(/[0-9]/, "Deve conter pelo menos um número")
```

```
.regex(/[^\a-zA-Z0-9]/, "Deve conter pelo menos um símbolo")
```

5. Crie um novo ficheiro **src/controllers/authController.ts** e adicione a lógica de verificação de identidade e emissão do Token:

```
import { type Request, type Response } from "express";
import bcrypt from "bcryptjs";
import jwt from "jsonwebtoken";
import * as userService from "../services/userService.js";

const JWT_SECRET = process.env.JWT_SECRET || "chave_super_secreta_aw";

export const login = async (req: Request, res: Response) => {
  const { email, password } = req.body;

  // 1. Delegar a consulta à base de dados para a camada de Serviço
  const user = await userService.findUserByEmail(email);

  // 2. Verificar se o utilizador existe e se a senha coincide com o Hash armazenado
  if (!user || !(await bcrypt.compare(password, user.password))) {
    return res.status(401).json({ error: "Credenciais inválidas" });
  }

  // 3. Gerar o JWT com o ID do utilizador no Payload
  const token = jwt.sign({ id: user.id }, JWT_SECRET, { expiresIn: "1h" });

  res.json({ token });
};
```

6. Atualize o userService.ts para conter uma função para pesquisar o utilizador pelo seu email:

```
// Função para pesquisa por email
export const findUserByEmail = async (email: string) => {
  return await prisma.user.findUnique({
    where: { email }
  });
};
```

7. Crie um authRoutes.ts que aponte para POST /api/auth/login . Deverá criar um novo router e alterar o necessário no server.ts para o suportar.

8. Crie a pasta **src/middlewares** e o ficheiro **authGuard.ts** para verificar o **Token** nos pedidos subsequentes:

```
import { Request, Response, NextFunction } from "express";
import jwt from "jsonwebtoken";

// 1. Criamos uma Interface estrita apenas para rotas protegidas
export interface AuthenticatedRequest extends Request {
  user?: { id: number; role?: string };
}

const JWT_SECRET = process.env.JWT_SECRET || "chave_super_secreta_aw";

export const authGuard = (
  req: AuthenticatedRequest,
  res: Response,
  next: NextFunction,
) => {
  const token = req.headers.authorization?.split(" ")[1];

  if (!token) {
    res.status(401).json({ error: "Acesso negado. Token em falta." });
    return;
  }

  try {
    const decoded = jwt.verify(token, JWT_SECRET) as {
      id: number;
      role?: string;
    };
    req.user = decoded;
    next();
  } catch (error) {
    res.status(403).json({ error: "Token inválido ou expirado." });
  }
};
```

9. Para que o Swagger permita enviar o Token, atualize o **options** no ficheiro **src/lib/swagger.ts**:

```
const options = {
  definition: {
    // ... configurações anteriores ...
```

```

components: {
  securitySchemes: {
    bearerAuth: {
      type: 'http',
      scheme: 'bearer',
      bearerFormat: 'JWT',
    }
  }
},
apis: ['./src/docs/*.yaml'],
};

```

10. Vamos agora proteger a rota de pesquisa pelo id do utilizador (GET /api/users/:id) para que seja acessível apenas a utilizadores autenticados com um token válido. Para isso acrescente o seguinte pedaço de código ao ficheiro **userController.ts**:

```

import { AuthenticatedRequest } from '../middlewares/authGuard.js';

export const getUserById = async (req: AuthenticatedRequest, res: Response) => {
  // 1. Validação Sintática (Zod)
  const result = userIdParamSchema.safeParse(req.params);
  if (!result.success) {
    return res.status(400).json({
      errors: result.error.issues.map((issue) => `${issue.path}: ${issue.message}`),
    });
  }

  // 2. Garantia de Autenticação
  if (!req.user) {
    return res.status(401).json({ error: "Utilizador não autenticado." });
  }

  // 3. Autorização
  const requestedId = parseInt(result.data.id as string);
  if (req.user.id !== requestedId) {
    return res.status(403).json({ error: "Acesso negado. Apenas pode consultar o seu próprio perfil." });
  }

  try {
    const user = await userService.getUserById(requestedId);
    if (!user) return res.status(404).json({ error: "Utilizador não encontrado." });
  }
}

```

```
// Projeção de Dados: Ocultar a Password
const { password, ...safeUser } = user;
res.status(200).json(safeUser);
} catch (error) {
  res.status(500).json({ error: "Erro interno do servidor." });
}
};
```

11. Por fim, precisamos de alterar o ficheiro de rotas **userRoutes.ts**, para incluir o middleware criado:

```
router.get('/:id', authGuard, getUserById);
```

12. No terminal, execute o script para ligar o servidor:

npx tsx watch src/server.ts

13. Realize vários pedidos de forma a verificar se os mecanismos de autenticação e proteção da password estão ativos
 - a. Registo: Crie um utilizador via POST `/api/users`. Verifique com um pedido get se a password é uma string ilegível (Hash).
 - b. Login: Submeta as credenciais corretas em POST `/api/auth/login`. Deverá receber um JSON com o campo token. Experimenta depois submeter um pedido com as credenciais incorretas.
 - c. Acesso Protegido: Tente aceder a uma rota protegida sem token (esperado: 401). Tente com um token aleatório (esperado: 403). Finalmente, use o botão Authorize no Swagger, insira o token, e verifique o acesso bem-sucedido. Poderá ser útil adicionar à rota no swagger (users.yaml) o campo:

```
security:
  - bearerAuth: []
```

3. Exercícios

Vamos utilizar os recursos Users, Books, Authors do guião anterior para adicionar segurança e autenticação na API nos exercícios seguintes.

Exercício 1

Utilize o middleware authGuard no seu bookRoutes.ts para garantir que apenas utilizadores autenticados podem criar ou apagar livros. As rotas de leitura (GET) devem permanecer públicas.

Dica: podem ser passados parâmetros adicionais para as funções das rotas como por exemplo `router.post('/', authGuard, createExampleController);`

Exercício 2

No seu ficheiro `books.yaml`, adicione o requisito de segurança à rota POST documentada. Adicione também as potenciais respostas de erro de autenticação. Dica: para adicionar requisitos de segurança na documentação é útil o uso da propriedade `security` com a subpropriedade `bearerAuth`.

`security:`
- `bearerAuth: []`

Exercício 3

Este exercício foca-se na prevenção de BOLA (Broken Object Level Authorization). Crie a estrutura necessária para suportar uma lista de desejos (Wishlist), onde cada utilizador pode guardar livros que pretende ler. Implemente a rota GET `/api/users/:id/wishlist`. O sistema deve garantir que um utilizador autenticado apenas consegue visualizar a sua própria lista. Se o utilizador ID 1 tentar aceder à lista do ID 2, a API deve responder com 403 Forbidden, mesmo que o token seja válido.

Exercício 4

Crie a estrutura necessária para suportar uma autorização baseada em cargos (roles). Para isso pode ser útil (1) adicionar um campo `role` ao User no Prisma, com o valor por omissão "USER"; (2) Criar um novo middleware `roleGuard.ts` que verifique se o utilizador autenticado possui a role "ADMIN"; (3) Aplicar este middleware à rota de eliminação de autores (DELETE `/api/authors/:id`). O sistema deve devolver um erro 403 Forbidden caso um utilizador normal tente realizar a operação. Pode ser útil atualizar o login no `authController` para que o jwt passe a considerar o role:

```
const token = jwt.sign({ id: user.id, role: user.role }, JWT_SECRET, { expiresIn: "1h" });
```

Exercício 5

Vamos adicionar uma rota que permitir o utilizador obter um novo token (refresh token) sem necessitar de realizar novo login. Para isso pode ser útil criar um endpoint `/api/auth/refresh` que, ao receber um token válido mas prestes a expirar, emita um novo token com uma nova validade, sem exigir a palavra-passe novamente. Valide o body desta requisição utilizando um esquema Zod

5. Tutoriais

Deve realizar os tutoriais de suporte abaixo e consultar a documentação oficial:

- [Helmet](#)
- [Bcrypt & Hashing](#)
- [JWT](#)
- [Express Rate Limit](#)